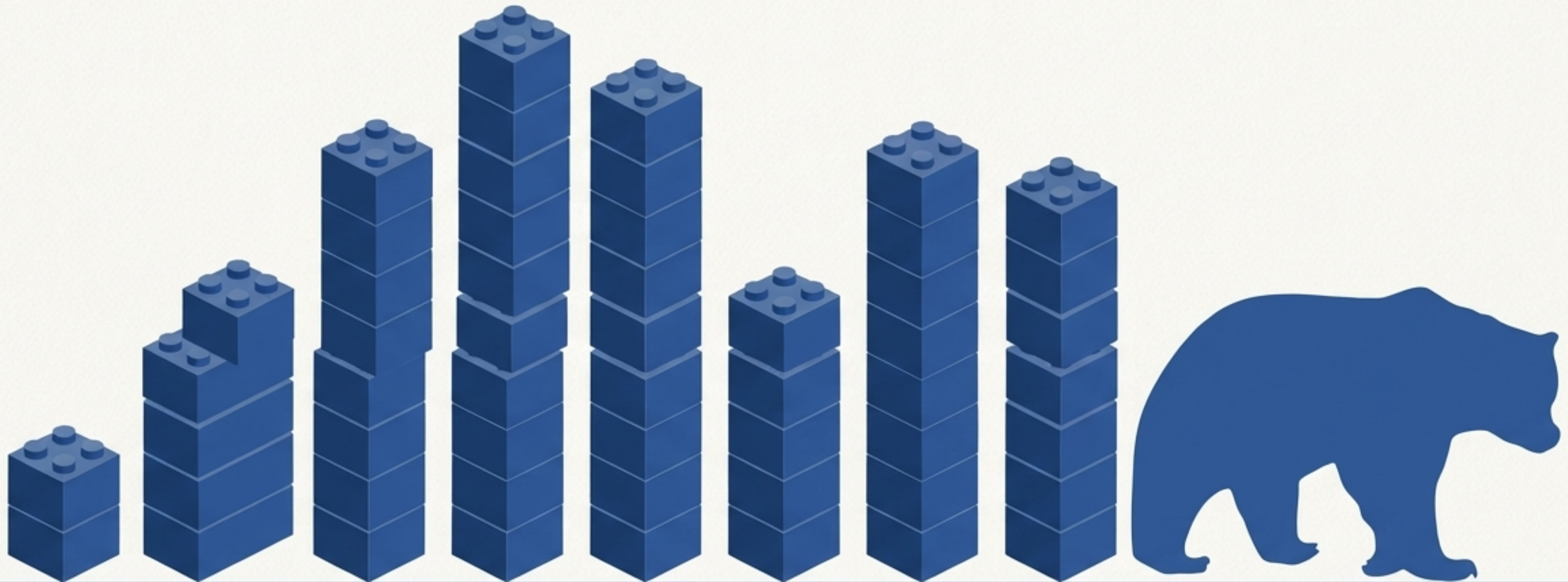


Construyendo con Legos de Datos

Una Introducción a las Estructuras Fundamentales de Pandas

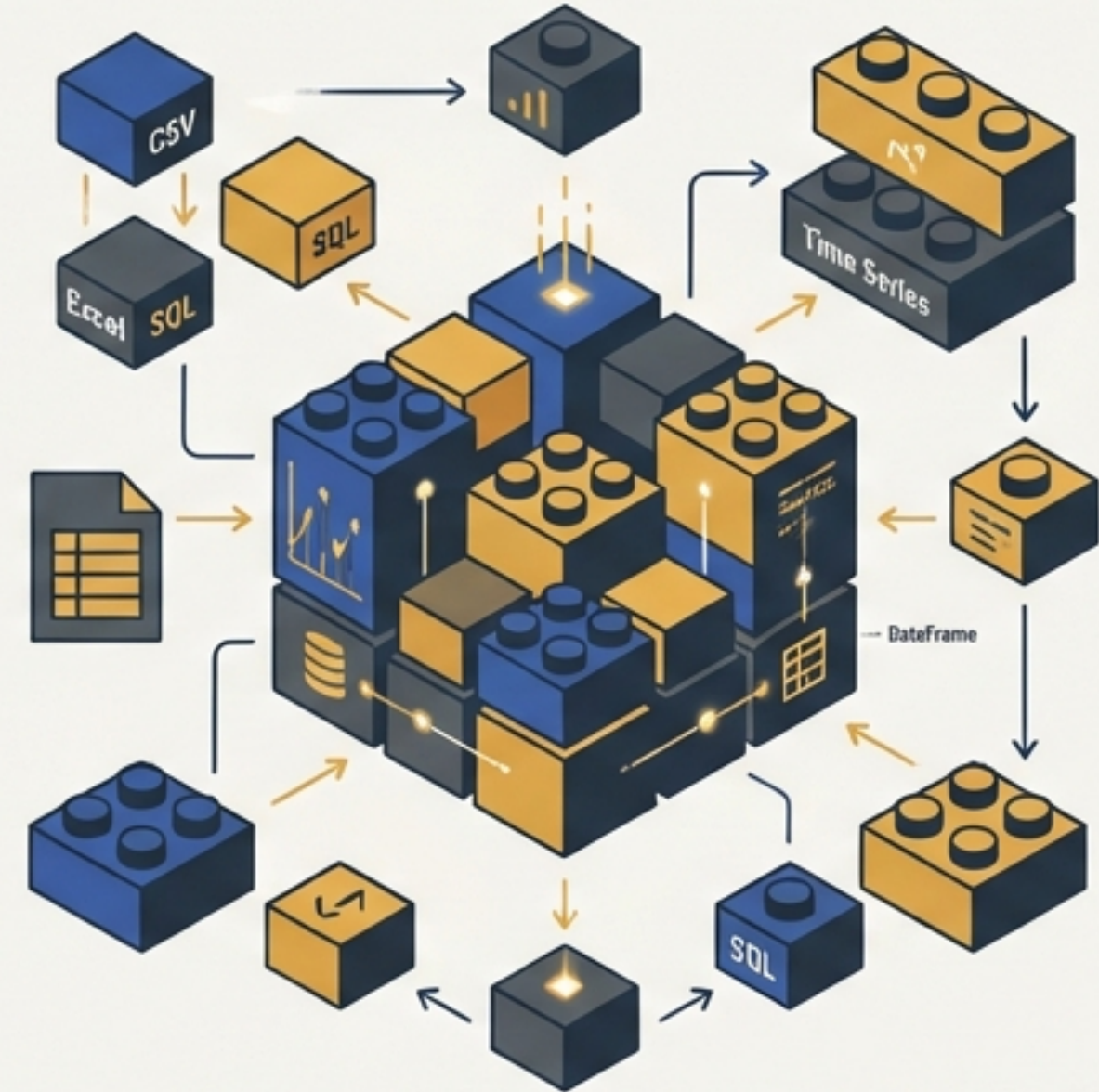


Pandas: La herramienta esencial para la manipulación de datos en Python.

Pandas es una librería de Python de código abierto, diseñada para ser rápida, potente, flexible y fácil de usar. Su objetivo es convertirse en la herramienta de análisis y manipulación de datos más potente y flexible disponible.

- Define nuevas estructuras de datos sobre NumPy: `Series` y `DataFrames`.
- Facilita la lectura y escritura de múltiples formatos (CSV, Excel, SQL).
- Permite acceso a datos mediante índices y etiquetas personalizadas.
- Ofrece métodos eficientes para reordenar, dividir y combinar datos.
- Optimizado para trabajar con series temporales.

``import pandas as pd`` es la convención estándar para empezar a trabajar.



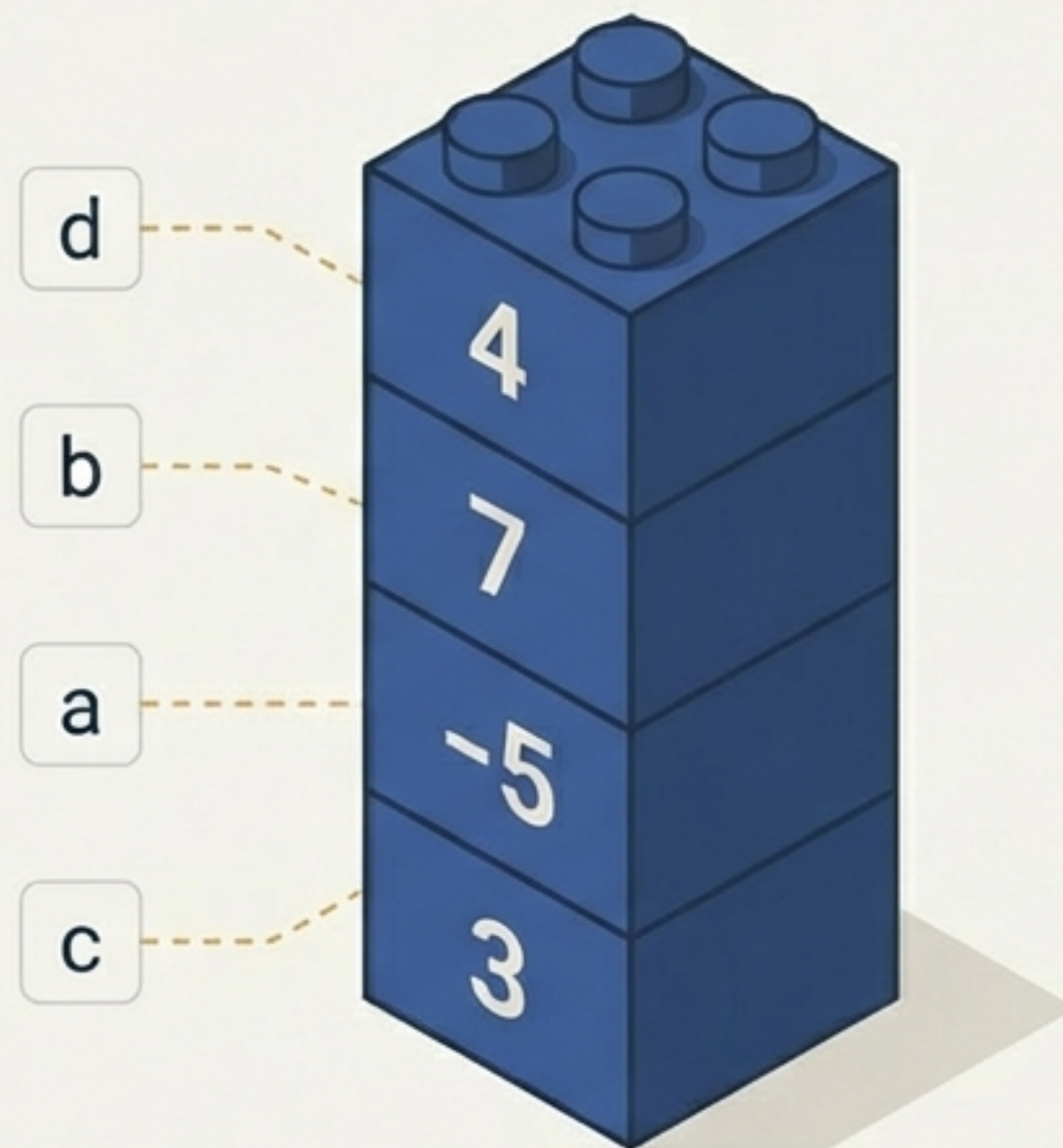
EL LADRILLO FUNDAMENTAL

La `Serie`: Más que datos, son datos con identidad.

Una `Serie` es un objeto unidimensional que contiene una secuencia de valores y un array asociado de etiquetas de datos, llamado **índice**.

Características Fundamentales:

1. **Homogénea**: Todos los elementos deben ser del mismo tipo de dato.
2. **Tamaño Inmutable**: La longitud de una `Serie` no se puede cambiar una vez creada, aunque su contenido sí.



Creando una `Serie`: Asignando etiquetas a nuestros datos.

Código (Python)

Ejemplo 1: Índice por defecto

```
import pandas as pd

# Si no se especifica un índice,
# se crea uno de 0 a N-1.
obj_simple = pd.Series([4, 7, -5, 3])
```

Ejemplo 2: Índice personalizado

```
# Un índice personalizado transforma los datos
# en información con contexto.
obj_con_indice = pd.Series([4, 7, -5, 3],
                           index=["d", "b", "a", "c"])
```

Resultado Visual (Terminal)

```
0      4
1      7
2     -5
3      3
dtype: int64
```

```
d      4
b      7
a     -5
c      3
dtype: int64
```


Una `Serie` se comporta como un diccionario ordenado.

La estructura de clave-valor de un diccionario se traduce directamente al par índice-valor de una `Serie`. Esta es una forma intuitiva y común de crear datos etiquetados.

Código (Python)

```
# Diccionario de poblaciones
dic = {"Ohio": 35000,
      "Texas": 71000,
      "Oregon": 16000,
      "Utah": 5000}

# Crear una Serie a partir del diccionario
serie_estados = pd.Series(dic)
```

Resultado Visual (Terminal)

```
Ohio      35000
Texas     71000
Oregon    16000
Utah       5000
dtype: int64
```

- También puedes convertir una `Serie` de vuelta a un diccionario con el método `.to_dict()`.

El mundo real es imperfecto: Gestionando datos ausentes con `NaN`.

Pandas representa los valores ausentes o faltantes con **`NaN`** (Not a Number). Esto ocurre cuando un índice no tiene un dato correspondiente.

Código (Python)

```
dic = {"Ohio": 35000, "Texas": 71000,  
       "Oregon": 16000, "Utah": 5000}  
estados = ["California", "Ohio", "Oregon", "Texas"]  
  
# "California" no está en el diccionario -> NaN  
# "Utah" no está en la lista de índices -> se omite  
serie_modif = pd.Series(dic, index=estados)  
  
# Detectar valores nulos  
pd.isna(serie_modif)
```

Resultados Visuales

****Salida de `serie\_modif`**:**

California	NaN
Ohio	35000.0
Oregon	16000.0
Texas	71000.0

dtype: float64

****Salida de `pd.isna(serie\_modif)`**:**

California	True
Ohio	False
Oregon	False
Texas	False

dtype: bool

`isna` devuelve `True` donde los datos son nulos.
`notna` hace lo contrario.

Dando contexto: Nombrando la `Serie` y su `Índice`.

Unos datos bien descritos son más fáciles de entender y utilizar. Los atributos `.name` y `.index.name` añaden metadatos cruciales.

Código (Python)

```
datos_altura = {"Pedro": 183, "Ana": 171,
                "Raquel": 157, "Nico": 165}
serie_altura = pd.Series(datos_altura)

# Asignar nombre a la Serie
serie_altura.name = "Altura_cm"

# Asignar nombre al Índice
serie_altura.index.name = "Nombres"

print(serie_altura)
```

Resultado Visual (Terminal)

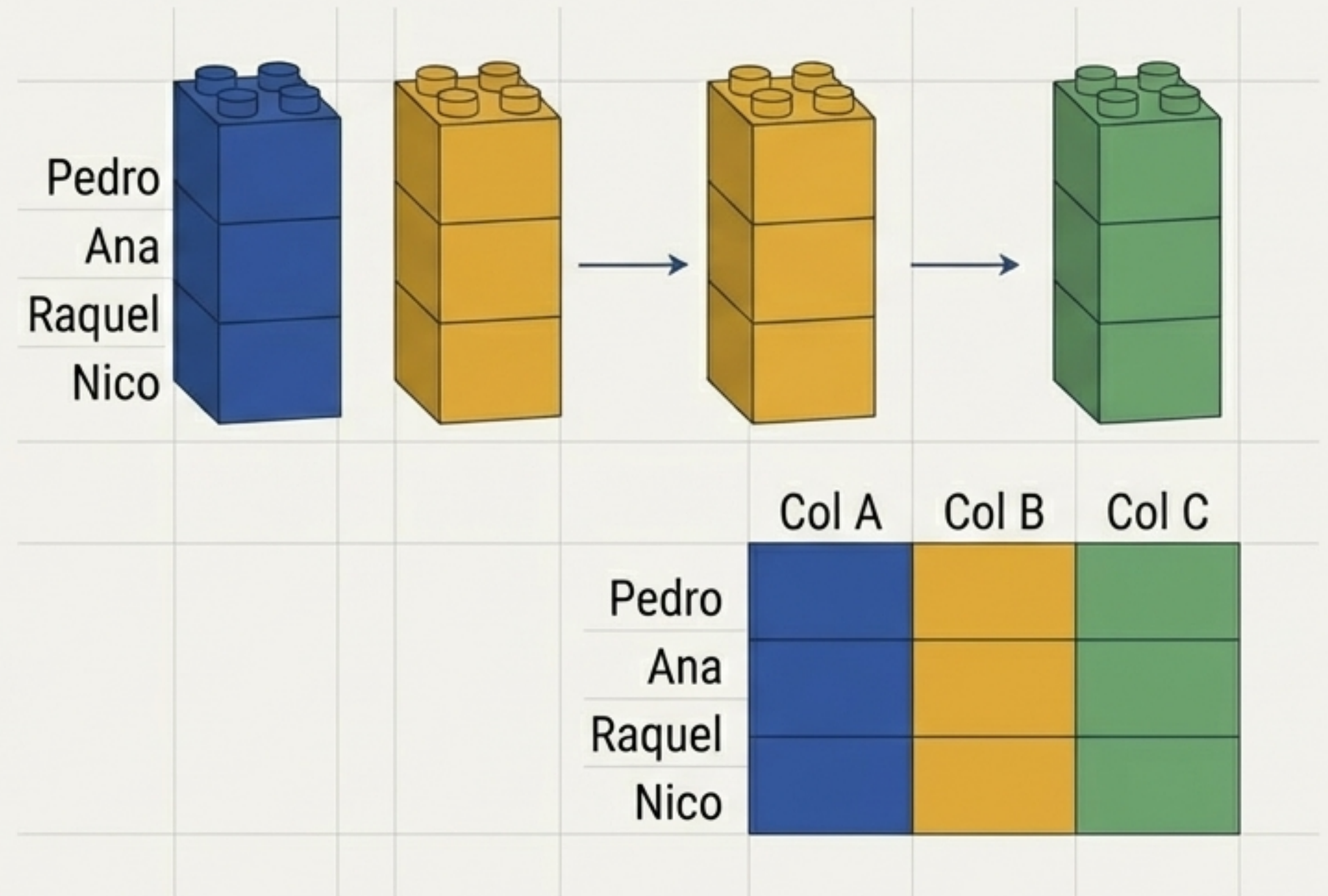
```
Nombres      .index.name
Pedro        183
Ana          171
Raquel       157
Nico         165
Name: Altura_cm, dtype: int64
               .name
```


ENSAMBLANDO LA ESTRUCTURA

El `DataFrame`: Una tabla de ladrillos `Serie`.

Un `DataFrame` representa una tabla rectangular de datos. Es una colección ordenada de columnas, donde cada columna puede tener un tipo de dato diferente.

Puedes pensar en un `DataFrame` como un diccionario de `Series`, donde todas las `Series` comparten el mismo índice.



Construyendo un `DataFrame` a partir de un diccionario de listas.

La forma más directa de crear un `DataFrame` es pasar un diccionario donde las claves serán los nombres de las columnas y las listas (o arrays) serán los datos de esas columnas.

Código (Python)

```
datos = {"ciudad": ["Almería", "Zafra",  
                  "Madrid", "Salamanca"],  
        "temp_max": [38, 40, 36, 32],  
        "temp_min": [12, -2, -8, -5]}  
  
frame = pd.DataFrame(datos)  
  
# También podemos nombrar los ejes  
frame.index.name = "Indice"  
frame.columns.name = "Características"  
  
print(frame)
```

Resultado Visual (Terminal)

Indice	Características	ciudad	temp_max	temp_min
0		Almería	38	12
1		Zafra	40	-2
2		Madrid	36	-8
3		Salamanca	32	-5

Definiendo la arquitectura: Orden de columnas y datos faltantes.

Tecnical es: la arquitetura: Se lixua la, editorial slilado el Inter Regular.

Código (Python)

Escenario 1: Especificar el orden de las columnas

```
# Los datos son los mismos del slide anterior...
datos = {"ciudad": [...], "temp_max": [...], "temp_min": [...]}

# Forzamos un orden de columnas específico
frame_ordenado = pd.DataFrame(datos,
                              columns=["temp_min", "temp_max", "ciudad"])
print(frame_ordenado)
```

Escenario 2: Pedir una columna que no existe

```
# Si una columna no está en los datos, se crea con valores NaN
frame_faltante = pd.DataFrame(datos,
                              columns=["temp_min", "humedad", "ciudad"])
print(frame_faltante)
```

Resultado Visual (Terminal)

Salida para `frame\_ordenado`

	temp_min	temp_max	ciudad
0	12	38	Almería
1	-2	40	Zafra
2	-8	36	Madrid
3	-5	32	Salamanca

Salida para `frame\_faltante`

	temp_min	humedad	ciudad
0	12	NaN	Almería
1	-2	NaN	Zafra
2	-8	NaN	Madrid
3	-5	NaN	Salamanca

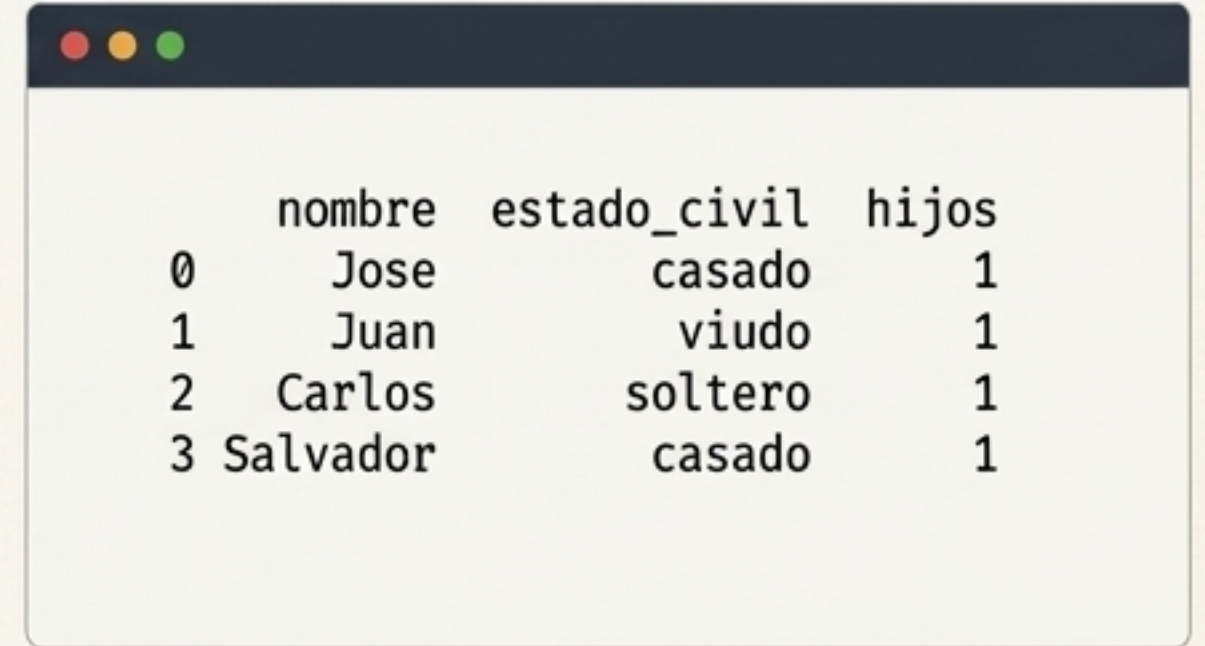
Añadiendo y modificando columnas: Expandiendo la estructura.

Las columnas de un DataFrame pueden ser asignadas y modificadas de forma flexible.

1. Crear una columna con un valor constante:

```
# Creamos un DataFrame inicial
datos = {"nombre":["Jose", "Juan", "Carlos", "Salvador"],
        "estado_civil":["casado", "viudo", "soltero", "casado"]}
frame = pd.DataFrame(datos, columns=["nombre", "estado_civil", "hijos"])

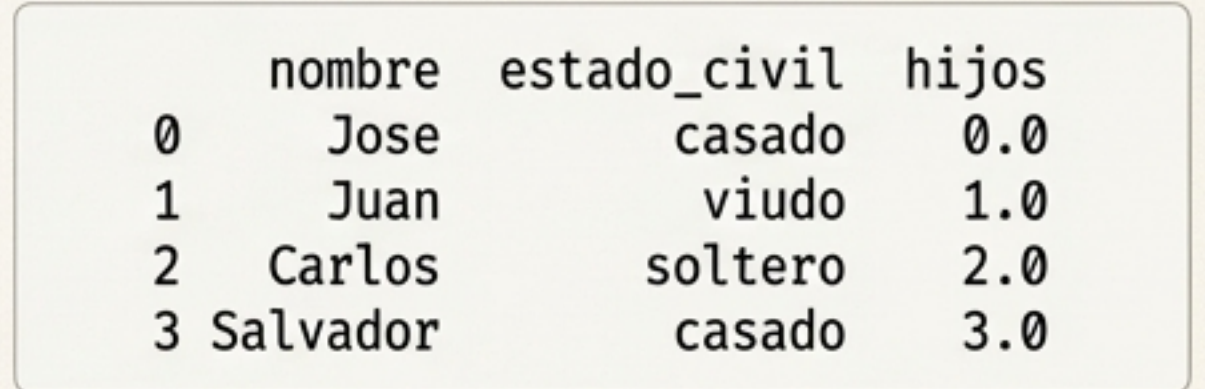
# Asignamos un valor a la columna 'hijos'
frame["hijos"] = 1
```



	nombre	estado_civil	hijos
0	Jose	casado	1
1	Juan	viudo	1
2	Carlos	soltero	1
3	Salvador	casado	1

2. Asignar desde un array (debe tener la misma longitud):

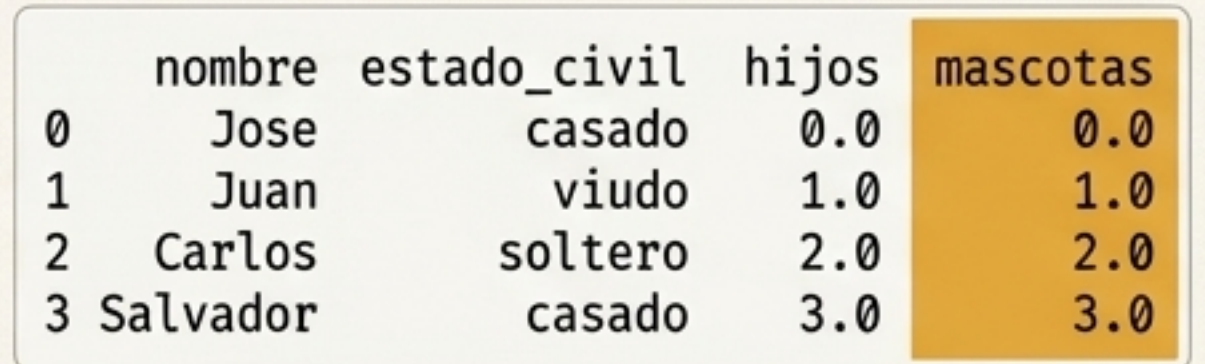
```
import numpy as np
frame["hijos"] = np.arange(4.)
```



	nombre	estado_civil	hijos
0	Jose	casado	0.0
1	Juan	viudo	1.0
2	Carlos	soltero	2.0
3	Salvador	casado	3.0

3. Añadir una nueva columna:

```
# Asignar una columna que no existe crea una nueva.
frame["mascotas"] = frame["hijos"]
```



	nombre	estado_civil	hijos	mascotas
0	Jose	casado	0.0	0.0
1	Juan	viudo	1.0	1.0
2	Carlos	soltero	2.0	2.0
3	Salvador	casado	3.0	3.0

Múltiples planos para construir un DataFrame.

El constructor de DataFrame es flexible y acepta diversos tipos de estructuras de datos de Python.

Tipo de estructura de datos	Conversión a DataFrame	Notas
ndarray de dos dimensiones	Se convierte directamente en un DataFrame	Puede incluir etiquetas opcionales para filas y columnas
Diccionario de arrays, listas o tuplas	Cada secuencia se convierte en una columna	Todas deben tener la misma longitud
Diccionario de Series	Cada Series se convierte en una columna	Los índices de las Series se combinan para formar el índice de las filas final
Diccionario de diccionarios	Cada diccionario interno se convierte en una columna	Las claves internas se convierten en índices de filas

LA BASE DE TODO

El `Índice`: La base inmutable de nuestras estructuras.

Los objetos `Índice` de Pandas contienen las **etiquetas** de los ejes (nombres de filas y columnas) y otros metadatos. Son el fundamento que alinea los datos.

Propiedad Crítica: Inmutabilidad

Un `Índice` no puede ser modificado una vez creado. Esto garantiza la integridad de los datos y previene errores accidentales.

Código que Falla

```
obj = pd.Series(range(3), index=["a", "b", "c"])
index = obj.index

# ¡Esto producirá un error!
index[1] = "d"
```

Resultado

TypeError: Index does not support mutable operations

```
TypeError                                Traceback (most recent call last)
<ipython-input-42-9387790323> in <module>
----> 1 index[1] = "d"
```


El `Índice` es una herramienta poderosa, no solo una etiqueta.

Un `Índice` se comporta como un conjunto de tamaño fijo y ofrece una variedad de métodos para operaciones de conjuntos y análisis.

Métodos y Propiedades Clave (Parte 1)		Métodos y Propiedades Clave (Parte 2)	
<code>append()</code>	Concatena con otros índices.	<code>insert()</code>	Devuelve un nuevo índice insertando un elemento.
<code>difference()</code>	Calcula la diferencia de conjuntos.	<code>is_monotonic</code>	Propiedad booleana (si está ordenado).
<code>intersection()</code>	Calcula la intersección (elementos comunes).	<code>is_unique</code>	Propiedad booleana (si no hay duplicados).
<code>union()</code>	Calcula la unión de conjuntos.	<code>unique()</code>	Devuelve un array con los valores únicos.
<code>isin()</code>	Comprueba si cada etiqueta está en una colección.		
<code>drop()</code>	Devuelve un nuevo índice eliminando valores especificados.		

Entender el `Índice` es fundamental para dominar la selección, alineación y combinación de datos en Pandas.